

**Faculdade de Informática e Administração Paulista**

Engenharia de Software

565065 - Augusto Barcelos Barros

555541 - Juan Francisco Alves Muradas

556197 - Caio Felipe de Lima Bezerra

555931 - Lucas Derenze Simidu

554873 - Sofia Fernandes

**Checkpoint 1- SOA**

**Arquitetura orientada a serviços**

São Paulo - SP

2026

# Documentação do Projeto – API de Aulas

## 1. Visão Geral do Projeto

Este projeto consiste em uma API desenvolvida em Java, com o objetivo de gerenciar informações relacionadas a aulas (como cadastro, listagem, atualização e exclusão).

A aplicação segue uma arquitetura em camadas, separando responsabilidades entre:

- Controller/Publicação (entrada da API)
- Service (regras de negócio)
- DAO (acesso a dados)
- DTO (estrutura dos dados)

## 2. Problema que o Projeto Resolve

O sistema foi desenvolvido para resolver a dificuldade de organização e gerenciamento de aulas, permitindo:

- Cadastro estruturado de aulas
- Consulta por diferentes critérios (ID, disciplina, dia da semana)
- Atualização de informações
- Exclusão de registros

Sem esse sistema, o controle dessas informações poderia ser feito de forma manual ou desorganizada, gerando:

- Falta de padronização
- Dificuldade de manutenção
- Risco de inconsistência de dados

## 3. Tecnologias Utilizadas

- Java

- Maven (gerenciamento de dependências – `pom.xml`)
- JDBC (acesso ao banco de dados)
- SQL (script em `create_table.sql`)
- SOAP (conceito) para comunicação
- CORS Filter para permitir acesso externo à API

## 4. Estrutura do Projeto

O projeto está organizado em pacotes, seguindo uma arquitetura em camadas que separa responsabilidades e facilita a manutenção do sistema.

Os principais pacotes são:

- **application**  
Responsável pela configuração e exposição da aplicação. Contém classes como o publicador da API e o filtro de CORS, que permite o acesso externo.
- **dao (Data Access Object)**  
Camada responsável pela comunicação direta com o banco de dados. Nela estão os métodos de inserção, consulta, atualização e exclusão de dados.
- **dto (Data Transfer Object)**  
Contém as classes que representam os dados manipulados pelo sistema, sendo utilizadas para transporte de informações entre as camadas.
- **service**  
Responsável pelas regras de negócio da aplicação. Atua como intermediário entre a camada de aplicação e a camada de acesso a dados.
- **factory**  
Responsável pela criação e gerenciamento da conexão com o banco de dados, centralizando essa configuração.
- **enums**  
Contém os tipos enumerados utilizados no sistema, garantindo padronização e evitando valores inválidos.
- **classe principal (Main)**  
Responsável por inicializar e executar a aplicação.

## 5. Funcionamento do Sistema

## **Fluxo da aplicação**

1. O cliente faz uma requisição (ex: cadastrar aula)
2. A requisição é recebida pelo Publicador
3. O Service (AulaService) processa as regras de negócio
4. O DAO (AulaDAO) acessa o banco de dados
5. Os dados são retornados ao cliente

## **6. Componentes Principais**

### **DTO – Aula.java**

Responsável por representar os dados da aula:

- Nome
- Disciplina
- Dia da semana
- Tipo
- Status

### **DAO – AulaDAO.java**

Responsável por:

- Inserir dados no banco
- Buscar registros
- Atualizar informações
- Deletar registros

### **Service – AulaService.java**

Camada onde ficam as regras de negócio:

- Validação de dados
- Regras antes de salvar ou atualizar
- Intermediação entre controller e DAO

### **Factory – ConnectionFactory.java**

Responsável por:

- Criar conexão com o banco de dados
- Centralizar configuração de acesso

## Enums

Padronizam valores do sistema:

- DiaDaSemana: Segunda, Terça, etc.
- StatusAula: Ativa, Cancelada, etc.
- TipoAula: Online, Presencial

## CORSFilter

Permite que a API seja acessada por aplicações externas (por exemplo, front-end).

## 7. Contexto de Implantação

Para executar o sistema é necessário:

1. Ter o Java instalado
2. Ter um banco de dados configurado
3. Executar o script:
  - a. `database/create_table.sql`
4. Configurar a conexão no arquivo ConnectionFactory
5. Executar o projeto pela classe Main.java

A API pode ser consumida por:

- Aplicações web (front-end)
- Aplicações mobile
- Ferramentas de teste como Postman (coleção disponível em /docs)

## 8. Problemas Resolvidos

- Centralização das informações de aulas
- Padronização dos dados com uso de enums
- Facilidade de manutenção
- Separação de responsabilidades (arquitetura em camadas)
- Facilidade de integração com outros sistemas

## 9. Boas Práticas Aplicadas

- Arquitetura em camadas (DAO, Service, DTO)
- Uso de interfaces (IAulaService)
- Separação de responsabilidades
- Uso de enums para evitar valores inválidos
- Centralização da conexão com banco de dados
- Organização em pacotes
- Uso de Maven para gerenciamento de dependências

## 10. Próximas Features (Melhorias Futuras)

- Implementação de autenticação (login)
- Geração de relatórios de aulas
- Filtros avançados (data, professor, etc.)
- Integração com front-end (React, Angular)
- Implementação de testes automatizados (JUnit)
- Deploy em nuvem (AWS, Azure)
- Documentação interativa com Swagger

## 11. Documentação da API

O projeto possui duas alternativas

### 1. API Docs Online:

[Corventures.github.io/SOA-CP1/](https://Corventures.github.io/SOA-CP1/)

## 2. Coleção de requisições na pasta feita no Bruno:

`/docs/FiapAulasBrunoCollection`

Essa coleção pode ser utilizada no Bruno para testar endpoints como:

- Cadastrar aula
- Listar todas
- Buscar por ID
- Atualizar
- Excluir

## 12. Conclusão

O projeto demonstra a construção de uma API bem estruturada, aplicando conceitos importantes de desenvolvimento backend, como:

- Organização em camadas
- Boas práticas de desenvolvimento
- Separação de responsabilidades
- Integração com banco de dados

Além disso, o sistema é escalável e permite evolução futura com facilidade.